

Теория языков программирования и методы трансляции

В рамках курса вам предстоит написать компилятор для языка Decaf. Decaf — это простой императивный язык, очень похожий на C.

В данном документе описываются синтаксис языка (то есть множество допустимых программ на Decaf) и его семантика (определение смысла каждой допустимой программы). Допускается вносить расширения, увеличивающие множество допустимых программ, при условии, что вы предоставите четкое описание новых принимаемых программ и их (корректной) семантики; такие расширения не должны включать синтаксически корректные программы на Decaf, которые явно запрещены семантическими правилами, изложенными в разделе 5. Примером допустимого расширения может служить введение новых типов данных или языковых конструкций. Запрещается вносить изменения, которые меняют семантику допустимых программ или исключают какие-либо допустимые программы из числа разрешенных.

1 Список изменений

25.09.2026 – первый вариант документ на основе переведенного MIT.

2 Лексические особенности

Все ключевые слова Decaf пишутся строчными буквами. Ключевые слова и идентификаторы чувствительны к регистру. Например,

`if` — это ключевое слово, а `IF` — идентификатор; `foo` и `Foo` — два разных идентификатора, обозначающих две различные переменные или метода.

Ключевые слова:

```
if bool break import continue else false for while
int long return len true void
```

Комментарии начинаются с `//` и заканчиваются в конце строки. Блочные комментарии начинаются с `/*` и заканчиваются соответствующим `*/`. Обращать вложенные блочные комментарии (например, `/* /* */ */`) не требуется.

Пробельные символы могут встречаться между любыми лексическими токенами (но не внутри отдельного токена). К пробельным символам относятся один или несколько пробелов, символов табуляции, символов перевода строки (возврат каретки, перевод строки, перевод страницы) и комментариев.

Ключевые слова и идентификаторы должны разделяться пробельными символами либо токеном, не являющимся ни ключевым словом, ни идентификатором. Например, `intfortrue` — это единый идентификатор, а не три отдельных ключевых слова. Если последовательность начинается с буквы или символа подчеркивания, то

она вместе с непосредственно следующей за ней максимально длинной последовательностью букв и цифр образует токен.

Строковые литералы состоят из символов `<char>`, заключенных в двойные кавычки. Символьный литерал состоит из символа `<char>`, заключенного в одинарные кавычки.

Если последовательность начинается с `0x`, то эти два символа и следующая за ними максимально длинная последовательность символов из набора `[0-9a-fA-F]` образуют литерал целого числа в шестнадцатеричной системе счисления. Если последовательность начинается с десятичной цифры (но не с `0x`), то следующая за ней максимально длинная последовательность десятичных цифр образует литерал целого числа в десятичной системе счисления. Длинная последовательность цифр (например, `123456789123456789123`) считается как единый токен.

Элемент `<char>` — это любой печатный символ ASCII (с десятичными кодами от 32 до 126 включительно), за исключением символов двойной кавычки (`"`), одинарной кавычки (`'`) и обратной косой черты (`\`), а также следующие двухсимвольные последовательности: `\"` (двойная кавычка), `\'` (одинарная кавычка), `\\` (обратная косая черта), `\t` (символ табуляции), `\n` (символ перевода строки), `\r` (символ возврата каретки) и `\f` (символ перевода страницы).

3 Грамматика

Обозначение	Значение
<code><foo></code>	означает, что <code>foo</code> является нетерминальным символом
<code>foo</code>	(выделено жирным шрифтом) означает, что <code>foo</code> — это терминал, то есть лексема или часть лексемы.
<code>[x]</code>	означает ноль или одно вхождение <code>x</code> , то есть <code>x</code> является необязательным; обратите внимание, что скобки в кавычках <code>'[]'</code> являются терминальными символами.
<code>x*</code>	означает ноль или более вхождений <code>x</code> .
<code>x+</code>	список одного или нескольких элементов <code>x</code> , разделенных запятыми.
<code>{ }</code>	Большие фигурные скобки используются для группировки; обратите внимание, что фигурные скобки в кавычках <code>'{ }'</code> являются терминалами.
<code> </code>	Разделяет варианты

$$\begin{aligned}
\langle \text{program} \rangle &\rightarrow \langle \text{import_decl} \rangle^* \langle \text{field_decl} \rangle^* \langle \text{method_decl} \rangle^* \\
\langle \text{import_decl} \rangle &\rightarrow \textbf{import} \langle \text{id} \rangle \textbf{;} \\
\langle \text{field_decl} \rangle &\rightarrow \langle \text{type} \rangle \{ \langle \text{id} \rangle \mid \langle \text{array_field_decl} \rangle \}^+, \textbf{;} \\
\langle \text{array_field_decl} \rangle &\rightarrow \langle \text{id} \rangle \textbf{'[}' \langle \text{int_literal} \rangle \textbf{']}' } \\
\langle \text{method_decl} \rangle &\rightarrow \{ \langle \text{type} \rangle \mid \textbf{void} \} \langle \text{id} \rangle (\left[\{ \langle \text{type} \rangle \langle \text{id} \rangle \}^+, \right]) \langle \text{block} \rangle \\
\langle \text{block} \rangle &\rightarrow \textbf{'\{'} \langle \text{field_decl} \rangle^* \langle \text{statement} \rangle^* \textbf{'\}} \\
\langle \text{type} \rangle &\rightarrow \textbf{int} \mid \textbf{long} \mid \textbf{bool} \\
\langle \text{statement} \rangle &\rightarrow \langle \text{location} \rangle \langle \text{assign_expr} \rangle \textbf{;} \\
&\mid \langle \text{method_call} \rangle \textbf{;} \\
&\mid \textbf{if} (\langle \text{expr} \rangle) \langle \text{block} \rangle [\textbf{else} \langle \text{block} \rangle] \\
&\mid \textbf{for} (\langle \text{id} \rangle = \langle \text{expr} \rangle \textbf{;} \langle \text{expr} \rangle \textbf{;} \langle \text{for_update} \rangle) \langle \text{block} \rangle \\
&\mid \textbf{while} (\langle \text{expr} \rangle) \langle \text{block} \rangle \\
&\mid \textbf{return} [\langle \text{expr} \rangle] \textbf{;} \\
&\mid \textbf{break} \textbf{;} \\
&\mid \textbf{continue} \textbf{;} \\
\langle \text{for_update} \rangle &\rightarrow \{ \langle \text{location} \rangle \langle \text{assign_expr} \rangle \} \\
\langle \text{assign_expr} \rangle &\rightarrow \langle \text{assign_op} \rangle \langle \text{expr} \rangle \mid \langle \text{increment} \rangle \\
\langle \text{assign_op} \rangle &\rightarrow = \mid += \mid -= \mid *= \mid /= \mid \% = \\
\langle \text{increment} \rangle &\rightarrow ++ \mid -- \\
\langle \text{method_call} \rangle &\rightarrow \langle \text{method_name} \rangle (\left[\langle \text{expr} \rangle^+, \right]) \\
&\mid \langle \text{method_name} \rangle (\left[\langle \text{extern_arg} \rangle^+, \right]) \\
\langle \text{method_name} \rangle &\rightarrow \langle \text{id} \rangle
\end{aligned}$$

$$\begin{aligned}
\langle \text{location} \rangle &\rightarrow \langle \text{id} \rangle \\
&| \langle \text{id} \rangle \text{ '[' } \langle \text{expr} \rangle \text{ ']' } \\
\\
\langle \text{expr} \rangle &\rightarrow \langle \text{location} \rangle \\
&| \langle \text{method_call} \rangle \\
&| \langle \text{literal} \rangle \\
&| \text{int } (\langle \text{expr} \rangle) \\
&| \text{long } (\langle \text{expr} \rangle) \\
&| \text{len } (\langle \text{id} \rangle) \\
&| \langle \text{expr} \rangle \langle \text{bin_op} \rangle \langle \text{expr} \rangle \\
&| - \langle \text{expr} \rangle \\
&| ! \langle \text{expr} \rangle \\
&| (\langle \text{expr} \rangle) \\
\\
\langle \text{extern_arg} \rangle &\rightarrow \langle \text{expr} \rangle \mid \langle \text{string_literal} \rangle \\
\\
\langle \text{bin_op} \rangle &\rightarrow \langle \text{arith_op} \rangle \mid \langle \text{rel_op} \rangle \mid \langle \text{eq_op} \rangle \mid \langle \text{cond_op} \rangle \\
\\
\langle \text{arith_op} \rangle &\rightarrow + \mid - \mid * \mid / \mid \% \\
\\
\langle \text{rel_op} \rangle &\rightarrow < \mid > \mid <= \mid >= \\
\\
\langle \text{eq_op} \rangle &\rightarrow == \mid != \\
\\
\langle \text{cond_op} \rangle &\rightarrow \&\& \mid || \\
\\
\langle \text{literal} \rangle &\rightarrow \langle \text{int_literal} \rangle \mid \langle \text{long_literal} \rangle \mid \langle \text{char_literal} \rangle \mid \langle \text{bool_literal} \rangle \\
\\
\langle \text{id} \rangle &\rightarrow \langle \text{alpha} \rangle \langle \text{alpha_num} \rangle^* \\
\\
\langle \text{alpha_num} \rangle &\rightarrow \langle \text{alpha} \rangle \mid \langle \text{digit} \rangle
\end{aligned}$$

$\langle \text{alpha} \rangle \rightarrow a \mid b \mid \dots \mid z \mid A \mid B \mid \dots \mid Z \mid -$
 $\langle \text{digit} \rangle \rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$
 $\langle \text{hex_digit} \rangle \rightarrow \langle \text{digit} \rangle \mid a \mid b \mid c \mid d \mid e \mid f \mid A \mid B \mid C \mid D \mid E \mid F$
 $\langle \text{int_literal} \rangle \rightarrow \{ \langle \text{decimal_literal} \rangle \mid \langle \text{hex_literal} \rangle \}$
 $\langle \text{long_literal} \rangle \rightarrow \{ \langle \text{decimal_literal} \rangle \mid \langle \text{hex_literal} \rangle \} \mathbf{L}$
 $\langle \text{decimal_literal} \rangle \rightarrow \langle \text{digit} \rangle \langle \text{digit} \rangle^*$
 $\langle \text{hex_literal} \rangle \rightarrow 0\mathbf{x} \langle \text{hex_digit} \rangle \langle \text{hex_digit} \rangle^*$
 $\langle \text{bool_literal} \rangle \rightarrow \mathbf{true} \mid \mathbf{false}$
 $\langle \text{char_literal} \rangle \rightarrow ' \langle \text{char} \rangle '$
 $\langle \text{string_literal} \rangle \rightarrow " \langle \text{char} \rangle^* "$

4 Семантика

Программа на языке Decaf состоит из одного файла. Она включает в себя объявления импорта, полей и методов. Объявления импорта делают доступными в файле Decaf методы из других библиотек. Объявления полей вводят переменные, к которым могут обращаться все методы программы. Объявления методов определяют функции или процедуры. Программа должна содержать объявление метода с именем `main`, имеющего тип `void` и не принимающего параметров. Выполнение программы на Decaf начинается с метода `main`.

4.1 Типы

В Decaf есть три основных типа: `int`, `long` и `bool`. Кроме того, существуют одномерные массивы целых чисел (`int[]`), длинных чисел (`long[]`) и логических значений (`bool[]`).

`int` — это 32-битное целое число со знаком. Тип `long` — это 64-битное целое число со знаком. Тип `bool` принимает значение `true` или `false` и при вызове внешних методов должен передаваться как 32-битное значение (1 или 0 соответственно).

Массивы могут быть объявлены в любой области видимости. Объявление массива должно содержать размер `[N]`. Все массивы являются одномерными и имеют фиксированный размер, определяемый на этапе компиляции. Индексация массивов осуществляется в диапазоне от 0 до `N - 1`, где `N > 0` — размер массива. Индексация массивов осуществляется с помощью обычных квадратных скобок `a[i]`.

Примечание для этапа 3. Вы могли заметить, что замена каждого вхождения `int` на эквивалентный 64-битный `long` даст семантически идентичную программу. Действительно, это допустимо на этапе генерации кода, если предположить, что и `int`, и `long` имеют 64-битную разрядность. Это приведет к существенному снижению производительности и усложнит некоторые виды оптимизации. Однако если команда сильно отстает от графика работы над проектом, такой подход позволит создать работающий компилятор.

4.2 Правила области видимости

В языке Decaf действуют простые и довольно строгие правила области видимости. Все идентификаторы должны быть определены (в тексте программы) до их использования. Например:

- Переменная должна быть объявлена до того, как она будет использована.
- Метод можно вызвать только из той части кода, которая следует за его заголовком (header). При этом допускается использование рекурсивных методов.

В любой точке программы на языке Decaf существуют как минимум две области видимости: глобальная область видимости и область видимости метода. Глобальная область видимости включает внешние методы, поля и методы, объявленные на верхнем уровне программы. Область видимости метода включает параметры метода, указанные в его объявлении. Дополнительные локальные области видимости существуют внутри каждого «блока» кода.

Идентификаторы, введенные в пределах определенной области видимости, доступны в этой области и во всех вложенных областях. Идентификаторы в более глубоко вложенной области перекрывают идентификаторы в менее глубоко вложенной области. Например, переменная, объявленная в методе, перекрывает метод с тем же именем, определенный в глобальной области видимости.

Один и тот же идентификатор не может быть определен более одного раза в пределах одной области видимости. Таким образом, имена полей и методов должны быть уникальными в глобальной области видимости, имена параметров метода — в области видимости метода, а имена локальных переменных — в области видимости конкретного блока. Аналогичным образом, идентификатор не может быть определен одновременно в списке параметров метода и в блоке верхнего уровня этого метода.

4.3 Ячейки памяти

Ячейки памяти в Decaf бывают глобальными (т. е. определенными на верхнем уровне программы) или локальными (т. е. определенными как параметры методов или локальные переменные). Каждая область хранения имеет определенный тип. Области типов `int`, `long` и `bool` содержат соответственно скалярные значения типов `integer`,

long, и bool. Области типов `int[]`, `long[]` и `bool[]` представляют собой массивы целых чисел (типов integers, longs) и логических значений соответственно.

Значения по умолчанию для типов `int`, `long` и `bool` неопределены. Аналогично, массивы изначально также неопределены.

Поскольку массивы в Decaf имеют фиксированный размер, глобальные массивы могут размещаться в области статических данных программы, и их не обязательно выделять в куче. Локальные массивы могут размещаться динамически в стеке или статически в куче, если это уместно.

4.4 Присваивание

Операция присваивания допустима только для скалярных ячеек памяти. В Decaf при присваивании используется семантика копирования значений.

Операция присваивания $\langle \text{location} \rangle = \langle \text{expr} \rangle$ копирует значение, полученное в результате вычисления $\langle \text{expr} \rangle$, в $\langle \text{location} \rangle$; она допустима только в том случае, если $\langle \text{location} \rangle$ и $\langle \text{expr} \rangle$ имеют одинаковый тип.

В Decaf реализована семантика явного приведения типов; неявное приведение типов не допускается. Поведение при переполнении в ходе приведения от типа большего размера к типу меньшего размера не определено. Вы можете реализовать любую разумную семантику (например, циклическое переполнение или насыщение).

Составная операция присваивания $\langle \text{location} \rangle += \langle \text{expr} \rangle$ увеличивает значение, хранящееся в $\langle \text{location} \rangle$, на величину $\langle \text{expr} \rangle$; она допустима только тогда, когда оба операнда — $\langle \text{location} \rangle$ и $\langle \text{expr} \rangle$ — имеют типы `int` и `long`.

Другие составные операторы присваивания — $\langle \text{location} \rangle -= \langle \text{expr} \rangle$, $\langle \text{location} \rangle *= \langle \text{expr} \rangle$, $\langle \text{location} \rangle /= \langle \text{expr} \rangle$ и $\langle \text{location} \rangle \%= \langle \text{expr} \rangle$ — работают аналогично $\langle \text{location} \rangle += \langle \text{expr} \rangle$, но выполняют соответственно вычитание, умножение, деление и вычисление остатка от деления.

Операции присваивания $\langle \text{location} \rangle ++$ и $\langle \text{location} \rangle --$ увеличивают и уменьшают значение, хранящееся в $\langle \text{location} \rangle$, на 1 соответственно; они применимы только к $\langle \text{location} \rangle$ типов `int` и `long`.

В выражениях присваивания, содержащих $\langle \text{expr} \rangle$ с побочными эффектами, сначала вычисляется левая часть, а затем — правая. Каждая из частей вычисляется ровно один раз и полностью до выполнения самого присваивания.

Следует отметить, что присваивание значения переменной-параметру метода внутри тела метода допустимо. Такие присваивания влияют только на область видимости внутри метода.

Запись в массив за пределами допустимого диапазона индексов приводит к **неопределенному** поведению.

4.5 Вызов метода и возврат значения

Вызов метода включает в себя (1) передачу значений параметров от вызывающей стороны к вызываемой, (2) выполнение тела вызываемого метода и (3) возврат к вызывающей стороне, возможно, с результатом.

Параметры методов передаются по значению (за исключением массивов и строк, передаваемых во внешние функции, что рассматривается в разделе **4.8**). Параметры метода считаются локальными переменными метода и инициализируются присваиванием значений, полученных в результате вычисления выражений с аргументами. Аргументы вычисляются слева направо.

Затем выполняется тело вызываемого метода — последовательно выполняются операторы его тела.

Метод, у которого не указан тип возвращаемого значения, может быть вызван только как оператор; его нельзя использовать в выражении. Такой метод возвращает управление вызывающей стороне при вызове **return** (выражение с результатом не допускается) или при достижении конца тела вызываемого метода.

Метод, возвращающий результат, может быть вызван как часть выражения, и в этом случае результатом вызова будет результат вычисления выражения в операторе **return** при достижении этого оператора. Недопустимо, чтобы управление достигло текстового конца метода, возвращающего результат.

Метод, возвращающий результат, может также вызываться как оператор. В этом случае результат игнорируется.

4.6 Операторы управления

4.6.1 **if**

Оператор **if** имеет обычную семантику. Сначала вычисляется значение $\langle \text{expr} \rangle$. Если результат равен **true**, выполняется первое условие. В противном случае **else** выполняется второе условие, если оно есть.

4.6.2 **while**

Оператор **while** имеет обычную семантику. Сначала вычисляется $\langle \text{expr} \rangle$. Если результат **false**, управление переходит к оператору, следующему за оператором **while**. В противном случае выполняется тело цикла. Если управление в операторе **while** доходит до конца тела цикла, оператор **while** выполняется снова.

4.6.3 **for**

Оператор **for** имеет обычную семантику, характерную для языка Си. $\langle \text{id} \rangle$ — это переменная индекса цикла, которая должна быть объявлена в области видимости как переменная типа **int** или **long**. Поскольку это должен быть идентификатор, это означает, что индексы массивов не могут быть допустимыми переменными индекса цикла. Перед входом в тело цикла ей присваивается значение первого $\langle \text{expr} \rangle$. Это выражение вычисляется один раз, непосредственно перед первым входом в цикл.

Второе "выражение" - это условие завершения цикла, которое должно быть преобразовано в тип `bool`. Оно вычисляется перед каждой итерацией тела цикла. Если результат равен `false`, управление переходит к инструкции, следующей за оператором `for`. В противном случае выполняется тело цикла. Обратите внимание, что `<expr>` может быть выражением с побочными эффектами (например, вызовом метода).

После каждой итерации тела цикла выполняется выражение обновления (`<for update>`). Поскольку оно может иметь побочные эффекты, его необходимо вычислять каждый раз. Обновляемая область памяти (`<location>`) не обязательно должна совпадать с переменной-счетчиком цикла. Затем управление возвращается к проверке условия завершения.

4.6.4 `break` и `continue`

Оператор `break` приводит к выходу из самого внутреннего цикла и передаче управления оператору, следующему за циклом. Оператор `continue` приводит к выходу из текущей итерации самого внутреннего цикла и передаче управления проверке условия в начале цикла для циклов `while` и к `<for update>` для циклов `for`.

4.6.5 `return`

Оператор `return` приводит к выходу из текущего метода и возвращению управления вызывающей стороне.

4.7 Выражения

Выражения подчиняются обычным правилам вычисления. Все бинарные операторы левоассоциативны.

Выражение с указанием местоположения вычисляется как значение, содержащееся в указанном месте.

Выражения вызова методов рассматриваются в разделе «*Вызов методов и возврат*». Операции с массивами рассматриваются в разделе «*Типы*». Выражения, связанные с вводом-выводом, рассматриваются в разделе «*Внешняя библиотека*».

Для отдельных выражений (т. е. вне циклов `if/while/for`) настоятельно рекомендуется использовать семантику короткого замыкания, но это не проверялось на практике. Это позволит привести вашу реализацию в соответствие с семантикой C.

Числовые литералы вычисляются как соответствующие им значения. Если после числового литерала стоит `L` то он вычисляется как тип `long`, а если нет — как тип `int`.

Символьные литералы преобразуются в целочисленные значения ASCII: например, `('A')` представляет собой целое число 65. Тип символьного литерала — `int`.

Оператор `len` вычисляет размер массива и возвращает значение типа `int`. Поскольку длина массива статична и известна на этапе компиляции, это константное выражение, которое может быть вычислено во время компиляции. Обратите внимание, что оператор `len` не является вызовом метода.

Арифметические операторы (<arith op> и унарный минус - имеют обычное значение. Оператор деления / вычисляет целую часть частного от деления операндов (например, 5/2 равно 2). Оператор взятия остатка от деления % вычисляет остаток от деления операндов (например, 5%2 равно 1). Деление и остаток от деления на ноль имеют неопределённое поведение и не будут проверяться. Результат арифметического оператора имеет тип `int` или `long`.

Операторы сравнения (<rel op>) используются для сравнения целочисленных выражений и имеют обычное значение. Результат работы оператора сравнения или оператора равенства имеет тип `bool`.

Операторы равенства == (равно) и != (не равно) определены только для типов `int`, `long` и `float`. Результат работы оператора равенства имеет тип `bool`.

Логические связки && и || интерпретируются с использованием метода вычисления с сокращением, как в Java. То есть правая часть выражения не выполняется (и, следовательно, не имеет побочных эффектов, если содержит вызов метода), если результат выражения уже известен (то есть если левая часть равна `false` для && или `true` для ||).

Логический оператор отрицания ! имеет обычное значение и определен только для выражений типа `bool`.

Логический оператор отрицания ! имеет обычное значение и определен только для выражений типа `bool`. Чтение данных из массива за пределами его границ приводит к неопределённому поведению.

Приведение типов осуществляется с помощью операторов `int()` и `long()`. Типы `int` и `long` можно привести друг к другу, но тип `bool` нельзя привести к числовому типу. Как уже упоминалось, семантика переполнения не определена.

Приоритет операторов от высшего к низшему:

	Операторы	Назначение
1	-	Унарный минус
2	!	Логическое отрицание
3	<code>int()</code> <code>long()</code>	Приведение типов
4	* / %	Умножение, деление, остаток от деления
5	+ -	Сложение, вычитание
6	< <= >= >	Операторы сравнения
7	== !=	Равенство
8	&&	Логическое И
9		Логическое ИЛИ

Обратите внимание, что этот приоритет не отражен в справочной грамматике.

4.8 Внешние библиотеки

В Dcaф есть метод вызова внешних функций, аналогичный тому, что используется в языке C. Внешняя функция помещается в область видимости с

помощью ключевого слова `import` в начале файла. Синтаксис (согласно грамматике) следующий:

`import` `<id>` ;

Все внешние функции обрабатываются так, как если бы они возвращали значение типа `int`. После объявления импорта их можно вызывать так же, как и любые другие функции. Единственное исключение — аргументы импорта могут содержать строковые литералы. Обычные методы Decaf не могут содержать строковые литералы в качестве аргументов.

4.8.1 Двоичный интерфейс приложения (Application Binary Interface, ABI) для внешних вызовов

Используйте 32 бита для Decaf `int`, 64 бита для Decaf `long` и 32 бита для Decaf `bool` (1 для `true`, 0 для `false`). Вы можете использовать любое разумное внутреннее представление типов в Decaf, но для корректной компоновки и выполнения вызовы внешних функций должны соответствовать этому ABI. Вы можете исходить из того, что для **внешних вызовов** потребуется не более 6 аргументов (то есть все аргументы поместятся в регистры).

4.8.2 Аргументы импорта

При вызове функции `import` к аргументам применяются следующие правила:

Выражения типа `bool`, `int`, `long` передаются как целые числа соответствующего размера. Выражения `bool[]`, `int[]` и `long []` передаются как указатели на первый элемент массива. Строковый литерал передается как указатель на первый символ массива символов с завершающим нулем. Возвращаемое значение передается как 32-битное целое число.

Пользователь функции `import` несет ответственность за то, чтобы переданные аргументы соответствовали сигнатуре функции, а возвращаемое значение использовалось только в том случае, если базовая библиотечная функция возвращает значение соответствующего типа. Аргументы передаются в функцию в соответствии со стандартным соглашением о вызовах системы. Компилятор не проверяет правильность количества и типа аргументов при импорте.

4.8.3 Внешние функции ввода-вывода

Помимо доступа к стандартной библиотеке C с помощью `import`, функцию ввода-вывода можно написать на C (или любом другом языке), скомпилировать с помощью стандартных инструментов, связать с системой времени выполнения и получить доступ к ней с помощью механизма `import`. Это единственный способ доступа к функциям ввода-вывода в Decaf.

5 Правила семантики

Эти правила накладывают дополнительные ограничения на набор допустимых программ на языке Decaf, помимо ограничений, предусмотренных грамматикой.

Программа, которая грамматически корректна и не нарушает ни одно из следующих правил, называется **допустимой**. Надежный компилятор будет явно проверять каждое из этих правил и выводить сообщение об ошибке при обнаружении нарушения. Надежный компилятор выводит как минимум одно сообщение об ошибке для каждой недопустимой программы и не выдает ошибок для допустимых программ.

1. Ни один идентификатор не объявляется дважды в одной и той же области видимости. Это правило распространяется и на идентификаторы импортируемых сущностей, существующих в глобальной области видимости.

2. Идентификатор не используется до его объявления.

3. Программа должна содержать определение метода с именем `main` имеющего тип `void` и не принимающего параметров. Следует учитывать, что, поскольку выполнение программы начинается с метода `main` любые методы, определенные после него, никогда не будут выполнены.

4. Количество и типы параметров при вызове метода (не являющегося импортируемым) должны совпадать с количеством и типами параметров, объявленных для этого указанными при объявлении этого метода.

5. Если вызов метода используется в качестве выражения, метод должен возвращать результат.

6. Строковые литералы и переменные-массивы не могут использоваться в качестве параметров методов (за исключением импортируемых методов).

(Примечание: выражение вида `a[0]` не является переменной-массивом).

7. Оператор `return` не должен содержать возвращаемого значения, если он находится не в теле метода, объявленного как возвращающий значение.

8. Тип выражения в операторе `return` должен совпадать с типом результата, объявленным для данного метода.

9. Идентификатор (`<id>`), используемый в качестве `<location>` (адресуемой области памяти), должен обозначать объявленную локальную или глобальную переменную либо параметр.

10. Идентификатор `<id>` в операторе вызова метода должен обозначать объявленный метод или импортируемую сущность.

11. Для всех конструкций вида `<id>[<expr>]:`

(a) `<id>` должен быть переменной-массивом, и

(b) тип выражения `<expr>` должен быть `int`.

12. Аргументом оператора `len` должна быть переменная-массив.

13. Выражение `<expr>` в операторах `if` или `while`, а также второе выражение `<expr>` в операторе `for`, должны иметь тип `bool`.

14. Операнды унарного минуса, а также операнды арифметических (`<arith op>`) и реляционных (`<rel op>`) операций должны иметь тип `int` или `long`. Операнды арифметических операций должны иметь одинаковый тип.

15. Операнды операций сравнения на равенство (`<eq op>`) должны иметь одинаковый тип: `int`, `long` или `bool`.

16. Операнды условных операций (`<cond op>`) и операнд операции логического отрицания `!` должны иметь тип `bool`.

17. Элемент `<location>` и выражение `<expr>` в операции присваивания (`<location> = <expr>`) должны иметь одинаковый тип.

18. Элемент `<location>` и выражение `<expr>` в операциях составного присваивания (`<location> += <expr>`, `<location> -= <expr>`, `<location> *= <expr>`, `<location> /= <expr>` и `<location> %= <expr>`) должны иметь тип `int` или `long`. То же самое относится к элементу `<location>` в операторах `++` и `--`.

19. Все операторы `break` и `continue` должны находиться внутри тела оператора `for` или `while`.

20. Операции приведения типов `int()` и `long()` должны принимать на вход только значения типа `int` или `long`:

(a) Конструкция `int(expr)` принимает выражение и преобразует его к типу `int`. Допускается, чтобы выражение `<expr>` имело тип `int`.

(b) Аналогичные правила применяются к конструкции `long(expr)` и типу `long`.

21. Все `int` литералы должны находиться в диапазоне $-2147483648 \leq x \leq 2147483647$ (32 бита).

22. Все литералы типа `long` должны находиться в диапазоне $-9223372036854775808 \leq x \leq 9223372036854775807$ (64 бита).

23. В качестве `<location>` (целевого объекта) в операции присваивания должна выступать скалярная переменная (не массив). (Примечание: выражение вида `a[0]` имеет скалярный тип).

6 Проверка во время выполнения

Помимо описанных выше ограничений, которые статически контролируются семантическим анализатором компилятора, следующие ограничения проверяются динамически. Генератор кода компилятора должен формировать инструкции для выполнения этих проверок; нарушения выявляются во время выполнения программы.

На 2026 год установлено только одно требование:

- Управление не должно выходить за пределы метода, объявленного как возвращающий результат.

При возникновении ошибки во время выполнения в терминал выводится соответствующее сообщение, и программа завершает работу. Если управление выходит за пределы метода, объявленного как возвращающий результат, программа

должна завершиться с кодом возврата -1. Выводимые сообщения об ошибках должны помогать программисту в поиске проблемы в исходном коде.